speckit.tasks

User Input

```
$ARGUMENTS
```

You **MUST** consider the user input before proceeding (if not empty).

Pre-Execution Checks

Check for extension hooks (before tasks generation): - Check if `.specify/extensions.yml` exists in the project root. - If it exists, read it and look for entries under the `hooks.before_tasks` key - If the YAML cannot be parsed or is invalid, skip hook checking silently and continue normally - Filter out hooks where `enabled` is explicitly `false`. Treat hooks without an `enabled` field as enabled by default. - For each remaining hook, do **not** attempt to interpret or evaluate hook `condition` expressions: - If the hook has no `condition` field, or it is null/empty, treat the hook as executable - If the hook defines a non-empty `condition`, skip the hook and leave condition evaluation to the HookExecutor implementation - For each executable hook, output the following based on its `optional` flag: - **Optional hook** (`optional: true`): ``` ## Extension Hooks

```
**Optional Pre-Hook**: {extension}
Command: `/{command}`
Description: {description}

Prompt: {prompt}
To execute: `/{command}`
```
```

- **Mandatory hook** (`optional: false`):

  ```
  ## Extension Hooks

  Automatic Pre-Hook: {extension}
  Executing: `/{command}`
  EXECUTE_COMMAND: {command}
  ```
  ```

```
Wait for the result of the hook command before proceeding to
the Outline.
```

After emitting the block above you MUST actually invoke the hook and wait for it to finish before continuing. Run it the same way you would run the command yourself in this agent/session (the invocation may differ from the literal `{command}` id shown above, e.g. a skills-mode agent runs it as `/skill:speckit-...` or `$speckit-...`). Emitting the block alone does not run the hook.

- If no hooks are registered or `.specify/extensions.yml` does not exist, skip silently

Outline

1. **Setup**: Run `.specify/scripts/bash/setup-tasks.sh --json` from repo root and parse FEATURE_DIR, TASKS_TEMPLATE, and AVAILABLE_DOCS list. `FEATURE_DIR` and `TASKS_TEMPLATE` must be absolute paths when provided. `AVAILABLE_DOCS` is a list of document names/relative paths available under `FEATURE_DIR` (for example `research.md` or `contracts/`). For single quotes in args like “I’m Groot”, use escape syntax: e.g ‘I’‘m Groot’ (or double-quote if possible: “I’m Groot”).

2. **Load design documents**: Read from FEATURE_DIR:

   - **Required**: plan.md (tech stack, libraries, structure), spec.md (user stories with priorities)
   - **Optional**: data-model.md (entities), contracts/ (interface contracts), research.md (decisions), quickstart.md (test scenarios)
   - **IF EXISTS**: Load `.specify/memory/constitution.md` for project principles and governance constraints
   - Note: Not all projects have all documents. Generate tasks based on what’s available.

3. **Execute task generation workflow**:

   - Load plan.md and extract tech stack, libraries, project structure
   - Load spec.md and extract user stories with their priorities (P1, P2, P3, etc.)
   - If data-model.md exists: Extract entities and map to user stories
   - If contracts/ exists: Map interface contracts to user stories
   - If research.md exists: Extract decisions for setup tasks
   - Generate tasks organized by user story (see Task Generation Rules below)
   - Generate dependency graph showing user story completion order

    - Create parallel execution examples per user story
    - Validate task completeness (each user story has all needed tasks, independently testable)

4. **Generate tasks.md**: Read the tasks template from TASKS_TEMPLATE (from the JSON output above) and use it as structure. If TASKS_TEMPLATE is empty, fall back to `.specify/templates/tasks-template.md`. Fill with:

    - Correct feature name from plan.md
    - Phase 1: Setup tasks (project initialization)
    - Phase 2: Foundational tasks (blocking prerequisites for all user stories)
    - Phase 3+: One phase per user story (in priority order from spec.md)
    - Each phase includes: story goal, independent test criteria, tests (if requested), implementation tasks
    - Final Phase: Polish & cross-cutting concerns
    - All tasks must follow the strict checklist format (see Task Generation Rules below)
    - Clear file paths for each task
    - Dependencies section showing story completion order
    - Parallel execution examples per story
    - Implementation strategy section (MVP first, incremental delivery)

Mandatory Post-Execution Hooks

You MUST complete this section before reporting completion to the user.

Check if `.specify/extensions.yml` exists in the project root. - If it does not exist, or no hooks are registered under `hooks.after_tasks`, skip to the Completion Report. - If it exists, read it and look for entries under the `hooks.after_tasks` key. - If the YAML cannot be parsed or is invalid, skip hook checking silently and continue to the Completion Report. - Filter out hooks where `enabled` is explicitly `false`. Treat hooks without an `enabled` field as enabled by default. - For each remaining hook, do **not** attempt to interpret or evaluate hook `condition` expressions: - If the hook has no `condition` field, or it is null/empty, treat the hook as executable - If the hook defines a non-empty `condition`, skip the hook and leave condition evaluation to the HookExecutor implementation - For each executable hook, output the following based on its `optional` flag: - **Mandatory hook** (`optional: false`) — **You MUST emit `EXECUTE_COMMAND:` for each mandatory hook**: ``` ## Extension Hooks

```
**Automatic Hook**: {extension}
Executing: `/{command}`
EXECUTE_COMMAND: {command}
```

```
After emitting the block above you MUST actually invoke the hook and wait for it to finish before continuing. Run it the same way you would run the command yourself in this agent/session (the invocation may differ from the literal `{command}` id shown above, e.g. a skills-mode agent runs it as `/skill:speckit-...` or `$speckit-...`). Emitting the block alone does not run the hook.
```

- **Optional hook** (`optional: true`):

  ```
  ## Extension Hooks

  **Optional Hook**: {extension}
  Command: `/{command}`
  Description: {description}

  Prompt: {prompt}
  To execute: `/{command}`
  ```

Completion Report

Output path to generated tasks.md and summary: - Total task count - Task count per user story - Parallel opportunities identified - Independent test criteria for each story - Suggested MVP scope (typically just User Story 1) - Format validation: Confirm ALL tasks follow the checklist format (checkbox, ID, labels, file paths)

Context for task generation: $ARGUMENTS

The tasks.md should be immediately executable - each task must be specific enough that an LLM can complete it without additional context.

Task Generation Rules

CRITICAL: Tasks MUST be organized by user story to enable independent implementation and testing.

Tests are OPTIONAL: Only generate test tasks if explicitly requested in the feature specification or if user requests TDD approach.

Checklist Format (REQUIRED)

Every task MUST strictly follow this format:

```
- [ ] [TaskID] [P?] [Story?] Description with file path
```

Format Components:

1. **Checkbox**: ALWAYS start with `- [ ]` (markdown checkbox)
2. **Task ID**: Sequential number (T001, T002, T003…) in execution order
3. **[P] marker**: Include ONLY if task is parallelizable (different files, no dependencies on incomplete tasks)
4. **[Story] label**: REQUIRED for user story phase tasks only
   - Format: [US1], [US2], [US3], etc. (maps to user stories from spec.md)
   - Setup phase: NO story label
   - Foundational phase: NO story label
   - User Story phases: MUST have story label
   - Polish phase: NO story label
5. **Description**: Clear action with exact file path

Examples:

- CORRECT: `- [ ] T001 Create project structure per implementation plan`
- CORRECT: `- [ ] T005 [P] Implement authentication middleware in src/middleware/auth.py`
- CORRECT: `- [ ] T012 [P] [US1] Create User model in src/models/user.py`
- CORRECT: `- [ ] T014 [US1] Implement UserService in src/services/user_service.py`
- WRONG: `- [ ] Create User model` (missing ID and Story label)
- WRONG: `T001 [US1] Create model` (missing checkbox)
- WRONG: `- [ ] [US1] Create User model` (missing Task ID)
- WRONG: `- [ ] T001 [US1] Create model` (missing file path)

Task Organization

1. **From User Stories (spec.md)** - PRIMARY ORGANIZATION:
   - Each user story (P1, P2, P3…) gets its own phase
   - Map all related components to their story:
     - Models needed for that story
     - Services needed for that story
     - Interfaces/UI needed for that story
     - If tests requested: Tests specific to that story
   - Mark story dependencies (most stories should be independent)
2. **From Contracts**:
   - Map each interface contract → to the user story it serves
   - If tests requested: Each interface contract → contract test task [P] before implementation in that story's phase
3. **From Data Model**:
   - Map each entity to the user story(ies) that need it
   - If entity serves multiple stories: Put in earliest story or Setup phase

    - Relationships → service layer tasks in appropriate story phase
4. **From Setup/Infrastructure**:
    - Shared infrastructure → Setup phase (Phase 1)
    - Foundational/blocking tasks → Foundational phase (Phase 2)
    - Story-specific setup → within that story's phase

Phase Structure

- **Phase 1**: Setup (project initialization)
- **Phase 2**: Foundational (blocking prerequisites - MUST complete before user stories)
- **Phase 3+**: User Stories in priority order (P1, P2, P3…)
    - Within each story: Tests (if requested) → Models → Services → Endpoints → Integration
    - Each phase should be a complete, independently testable increment
- **Final Phase**: Polish & Cross-Cutting Concerns

Done When

- [ ] tasks.md generated with all phases, task IDs, and file paths
- [ ] Extension hooks dispatched or skipped according to the rules in Mandatory Post-Execution Hooks above
- [ ] Completion reported to user with task count, story breakdown, and MVP scope